

Conceptos Fundamentales de la POO

Para entender este paradigma, es esencial familiarizarse con los siguientes conceptos:

1. Clases y Objetos

- **Clase:** Es una plantilla o molde que define la estructura general de un objeto (qué datos y qué acciones tendrá).
- **Objeto:** Es una instancia específica creada a partir de una clase. Si la clase es "Auto", un objeto podría ser "Auto rojo" o "Auto deportivo".

2. Atributos y Métodos

- **Atributos:** Son las características o datos que definen el estado de un objeto (ej. color, marca, modelo).
- **Métodos:** Son las acciones o comportamientos que el objeto puede realizar (ej. acelerar, frenar).

3. Los Cuatro Pilares de la POO

La POO se basa en cuatro principios fundamentales que rigen la arquitectura del código:

- **Abstracción:** Consiste en aislar la información esencial de un objeto y suprimir los detalles innecesarios. Se enfoca en el "qué" hace el objeto, no en el "cómo".
- **Encapsulación:** Es el mecanismo que agrupa los datos y los métodos dentro del mismo objeto y oculta o restringe el acceso a su estado interno. Esto protege los datos de modificaciones indebidas.
- **Herencia (o Jerarquía):** Permite que una clase (hija) adquiera los atributos y métodos de otra clase existente (padre). Esto fomenta la reutilización del código y evita la redundancia.
- **Polimorfismo:** Es la capacidad de que diferentes objetos respondan de manera distinta a la misma instrucción. Por ejemplo, si llamas al método "hablar" en un objeto "Perro", este ladrará, mientras que en un objeto "Gato", maullará.

3.1 Abstracción

Como la propia palabra indica, el principio de abstracción lo que implica es que la clase debe representar las características de la entidad hacia el mundo exterior, pero ocultando la complejidad que llevan aparejada. O sea, nos abstrae de la complejidad que haya dentro dándonos una serie de atributos y comportamientos (propiedades y funciones) que podemos usar sin preocuparnos de qué pasa por dentro cuando lo hagamos.

Así, una clase (y por lo tanto todos los objetos que se crean a partir de ella) debe exponer para su uso solo lo que sea necesario. Cómo se haga "por dentro" es irrelevante para los programas que hagan uso de los objetos de esa clase.

3.2 Encapsulación

Es la característica de un lenguaje POO que permite que todo lo referente a un objeto quede aislado dentro de éste. Es decir, que todos los datos referentes a un objeto queden "encerrados" dentro de éste y sólo se puede acceder a ellos a través de los miembros que la clase proporcione (propiedades y métodos).

Gracias a la encapsulación, toda la información de un objeto está contenida dentro del propio objeto.

3.3 Herencia

Desde el punto de vista de la genética, cuando una persona obtiene de sus padres ciertos rasgos (el color de los ojos o de la piel, una enfermedad genética, etc...) se dice que los hereda. Del mismo modo en POO cuando una clase hereda de otra obtiene todos los rasgos que tuviese la primera.

Dado que una clase es un patrón que define cómo es y cómo se comporta una cierta entidad, una clase que hereda de otra obtiene todos los rasgos de la primera y añade otros nuevos y además también puede modificar algunos de los que ha heredado.

A la clase de la que se hereda se le llama clase base, y a la clase que hereda de ésta se le llama clase derivada.

3.4 Polimorfismo

La palabra polimorfismo viene del griego "polys" (muchos) y "morfo" (forma), y quiere decir "cualidad de tener muchas formas".

En POO, el concepto de polimorfismo se refiere al hecho de que varios objetos de diferentes clases, pero con una base común, se pueden usar de manera indistinta, sin tener que saber de qué clase exacta son para poder hacerlo.

El polimorfismo nos permite utilizar a los objetos de manera genérica, aunque internamente se comporten según su variedad específica.

Principios POO en C#

Abstracción en C#

Es uno de los cuatro pilares de la Programación Orientada a Objetos (POO).

Permite ocultar los detalles de implementación complejos y mostrar solo la funcionalidad esencial mediante clases abstractas e interfaces. Se utiliza para definir contratos y crear bases reutilizables.

Clases Abstractas

Una clase abstracta se declara con la palabra clave **abstract** y sirve como plantilla base.

No se puede instanciar directamente; debe ser heredada.

Puede contener métodos con implementación propia y métodos abstractos que **obligatoriamente** deben ser implementados por las clases derivadas.

```
public abstract class Animal
{
    // Método implementado
    public void Respirar()
    {
        Console.WriteLine("Inhalar y exhalar...");
    }

    // Método abstracto (sin cuerpo, se debe implementar en la clase hija)
    public abstract void EmitirSonido();
}

public class Perro : Animal
```

```
{
    public override void EmitirSonido()
    {
        Console.WriteLine("¡Guau!");
    }
}
```

Interfaces

A diferencia de las clases abstractas, las interfaces actúan como un contrato puro. Una clase puede implementar múltiples interfaces, logrando una forma de herencia múltiple. A partir de C# 8.0, también pueden incluir una implementación predeterminada para sus miembros.

```
public interface IVolador
{
    void Volar();
}

public class Pato : Animal, IVolador
{
    public override void EmitirSonido()
    {
        Console.WriteLine("¡Cuack!");
    }

    public void Volar()
    {
        Console.WriteLine("El pato está volando.");
    }
}
```

Para acceder a explicaciones detalladas y guías paso a paso sobre cómo implementar estos conceptos en tus proyectos, puedes consultar el tutorial de abstracción en Oregoom.com

Encapsulación en C#

La encapsulación en C# consiste en ocultar el estado interno de un objeto y proteger sus datos. Se logra declarando los campos como privados (**private**) y controlando su acceso y modificación exclusivamente a través de propiedades públicas o métodos.

¿Por qué es útil?

- **Protección de datos:** Evita que código externo modifique variables de forma accidental o inválida (por ejemplo, asignar una edad negativa).
- **Flexibilidad:** Permite cambiar la estructura interna de una clase sin romper el código de otros programas que la utilizan.
- **Control:** Se puede agregar lógica (validaciones) dentro de los bloques get y set.

Ejemplo

En este ejemplo, la variable saldo está oculta y protegida. Solo se puede alterar a través del método Depositar o leer mediante la propiedad Saldo, lo que evita saldos negativos.

```
public class CuentaBancaria
{
    // Campo privado: oculto al exterior
    private decimal saldo;

    // Propiedad pública: permite acceder al saldo de forma controlada
    public decimal Saldo
    {
        get { return saldo; }
    }

    // Método para modificar el dato con validación
    public void Depositar(decimal monto)
    {
        if (monto > 0)
        {
            saldo += monto;
        }
    }
}
```

Modificadores de Acceso

El encapsulamiento se apoya en palabras clave que determinan quién puede ver o usar cada parte del código:

- **private:** Solo accesible dentro de la misma clase. Es el nivel predeterminado para los campos.
- **public:** Accesible desde cualquier parte del programa.
- **protected:** Accesible dentro de la misma clase y en clases que hereden de ella.
- **internal:** Accesible únicamente dentro del mismo ensamblado o proyecto.

Un ensamblado (assembly) es el bloque de construcción fundamental de las aplicaciones .NET.

Es un archivo compilado (con extensión .dll o .exe) que contiene el código en Lenguaje Intermedio (CIL) y metadatos, actuando como la unidad básica de despliegue, control de versiones, reutilización y seguridad.

Componentes principales de un ensamblado

Un ensamblado se compone de las siguientes partes clave:

- **Manifiesto:** Un bloque de metadatos que contiene el nombre del ensamblado, la versión, los tipos que expone y una lista de otros ensamblados de los que depende.
- **Metadatos:** Información descriptiva sobre los tipos, clases y miembros contenidos en el archivo, lo que permite que el código sea autodescriptivo y compatible con la técnica de Reflection.
- **Código CIL (Common Intermediate Language):** El código fuente en C# se compila primero a este lenguaje intermedio, el cual será traducido a código de máquina por el compilador Just-In-Time (JIT) en el momento de la ejecución.

- **Recursos:** Archivos adicionales que la aplicación pueda necesitar, como imágenes, texto o archivos de configuración.

Tipos de ensamblados

- **Ensamblados privados:** Diseñados para ser utilizados únicamente por una sola aplicación y ubicados en el mismo directorio que el archivo ejecutable.
- **Ensamblados compartidos:** Diseñados para ser utilizados por múltiples aplicaciones y almacenados globalmente en la Caché Global de Ensamblados (GAC), lo que permite manejar diferentes versiones simultáneamente.

[Mas info sobre ensamblados](#)

Herencia en C#

La herencia en C# permite crear nuevas clases a partir de otras existentes. La clase hija hereda todos los miembros (atributos y métodos) de la clase padre (clase base) y puede añadir los suyos propios.

En C# solo existe la **herencia simple**, por lo que una clase solo puede tener una clase base directa.

Sintaxis básica

Para que una clase herede de otra, se utiliza el símbolo de dos puntos : seguido del nombre de la clase padre.

```
// Clase Padre (Clase base)
public class Persona
{
    public string Nombre { get; set; }
    public int Edad { get; set; }

    public void Saludar()
    {
        Console.WriteLine($"Hola, soy {Nombre}");
    }
}

// Clase Hija (Clase derivada)
public class Empleado : Persona
{
    public decimal Salario { get; set; }
}
```

En este ejemplo, Empleado hereda Nombre, Edad y Salario.

Constructores y la palabra clave base

Cuando la clase base tiene un constructor con parámetros, la clase hija debe implementar su propio constructor y utilizar la palabra reservada base para enviar los datos necesarios a la clase padre.

```
public class Empleado : Persona
{
    public decimal Salario { get; set; }

    // El constructor de Empleado llama al constructor de Persona usando 'base'
    public Empleado(string nombre, int edad, decimal salario) : base(nombre, edad)
    {
        Salario = salario;
    }
}
```

Polimorfismo: Virtual y Override

Se puede modificar el comportamiento de un método heredado utilizando virtual en la clase base y override en la clase derivada.

```
public class Persona
{
    public virtual void Saludar()
    {
        Console.WriteLine("Hola, soy una persona.");
    }
}

public class Empleado : Persona
{
    public override void Saludar()
    {
        Console.WriteLine("Hola, soy un empleado.");
    }
}
```

Ejemplos

Polimorfismo en C#

El polimorfismo es un pilar de la Programación Orientada a Objetos (POO) que permite tratar objetos de diferentes clases derivadas como si fueran de una clase base común.

Esto significa que, al llamar a un mismo método, cada objeto responde de forma única y especializada.

En C#, existen dos tipos principales de polimorfismo:

1. Polimorfismo en tiempo de compilación (Estático)

Se logra a través de la sobrecarga. Consiste en definir varios métodos con el mismo nombre dentro de la misma clase, pero con diferentes parámetros (número o tipo).

El compilador decide qué método ejecutar según los argumentos enviados.

2. Polimorfismo en tiempo de ejecución (Dinámico)

Es el más potente y se logra mediante la herencia y la sobrescritura de métodos.

Permite que el programa decida en el momento en que se ejecuta (runtime) qué versión de un método debe llamar.

Para implementarlo, se usan dos palabras clave fundamentales:

- **virtual:** Se coloca en el método de la clase base para indicar que puede ser modificado.
- **override:** Se coloca en el método de la clase derivada para reemplazar el comportamiento del método base.

Ejemplo Práctico

Tenemos una clase base Vehiculo y clases derivadas como Coche y Bicicleta. Cada una necesita arrancar de manera diferente.

```
public class Vehiculo
{
    public virtual void EncenderMotor()
    {
        Console.WriteLine("El vehículo está encendido.");
    }
}

public class Coche : Vehiculo
{
    public override void EncenderMotor()
    {
        Console.WriteLine("El coche ruge al encender.");
    }
}

public class Bicicleta : Vehiculo
{
    public override void EncenderMotor()
    {
        Console.WriteLine("No tengo motor, ¡empiezo a pedalear!");
    }
}
```

Al usar polimorfismo, se pueden almacenar todos los vehículos en una colección de tipo Vehiculo y llamarlos a todos por igual, sin preocuparnos por el tipo específico en ese momento.

```
List<Vehiculo> misVehiculos = new List<Vehiculo>();
misVehiculos.Add(new Coche());
misVehiculos.Add(new Bicicleta());

foreach(Vehiculo v in misVehiculos)
{
    v.EncenderMotor();
    // Imprime: "El coche ruge al encender." y "No tengo motor, ¡empiezo a pedalear!"
}
```

